

ΑΝΑΚΤΗΣΗ ΠΛΗΡΟΦΟΡΙΩΝ ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ 2026

ΑΓΓΕΛΟΣ ΜΕΝΓΚΑ Ε22102

Πρόβλημα:

“ Δίνεται μεγάλο σύνολο διανυσματικών δεδομένων S που περιέχει n -διάστατα διανύσματα

και ζητείται η υλοποίηση αλγόριθμου που δοθέντος ενός διανύσματος-ερώτησης q υπολογίζει τα k κοντινότερα διανύσματα βάσει ευκλείδειας απόσταση $d(q, s_i)$ όπου $s_i \in S$, δηλαδή επιστρέφει το

υποσύνολο διανυσμάτων R :

$$R = \{s_i \in S : d(q, s_i) \leq d(q, s_j) \text{ για κάθε } s_i \in R \text{ και } s_j \in S - R\}, |R|=k$$

Για την υλοποίηση των αλγορίθμων θα χρησιμοποιηθεί μια προσέγγιση που στηρίζεται στη χρήση

αντεστραμμένων ευρετηρίων (inverted indexes) ”

ΥΛΟΠΟΙΗΣΗ ΚΑΙ ΑΛΓΟΡΙΘΜΟΙ

Δεδομένα

Δίνεται το σύνολο δεδομένων SIFT για την επίλυση του προβλήματος. Το dataset περιέχει 4 σύνολα:

- Sift base: Το σύνολο δεδομένων στο οποίο θα γίνεται η αναζήτηση όταν ο χρήστης εισάγει νέο ερώτημα (Στη περιγραφή προβλήματος το σύνολο αυτό αναφέρεται ως S)
- Sift Train: Το σύνολο δεδομένων που θα χρησιμοποιηθεί για την δημιουργία του ανεστραμμένου ευρετηρίου
- Sift query: Το σύνολο δεδομένων που περιέχει όλα τα ερωτήματα χρηστών. (Στη περιγραφή προβλήματος το σύνολο αυτό αναφέρεται ως Q)
- Sift ground truth: Τα πραγματικά K κοντινότερα διανύσματα για κάθε Q . Με αυτό το σύνολο θα συγκρίνουμε τα αποτελέσματα των αλγορίθμων

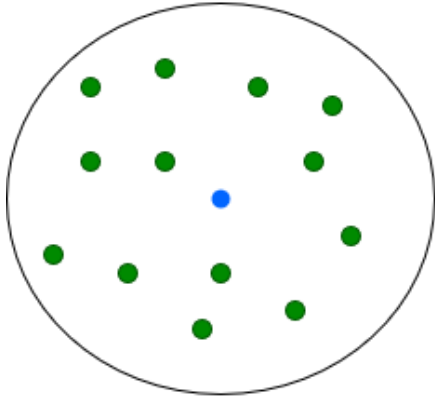
Μέγεθος Συνόλων δεδομένων

```
length of S 1000000  
length of train set 100000  
length of Q 10000  
length of Ground truth 10000
```

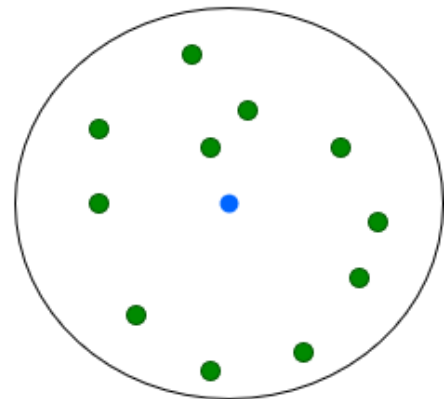
Δημιουργία αντεστραμμένου ευρετηρίου

Για τη δημιουργία του inverted index χρησιμοποιήθηκε ο αλγόριθμος Kmeans για την δημιουργία συστάδων στον διανυσματικό χώρο. Υποθέτουμε ότι κάθε συστάδα αντιστοιχεί με έναν όρο ενός λεξικού και τα διανύσματα των συστάδων αντιστοιχούν με Document IDs που τον περιέχουν. Δημιουργούμε $\sqrt{N}$ συστάδες (από κεφάλαιο 05 Πλήρες Σύστημα Ανάκτησης Πληροφοριών και Αξιολόγηση Μηχανών Αναζήτησης Πολύ πιθανώς: "κάθε αρχηγός έχει $\sim \sqrt{N}$ followers", όπου κάθε αρχηγός είναι ένα κέντρο συστάδας)

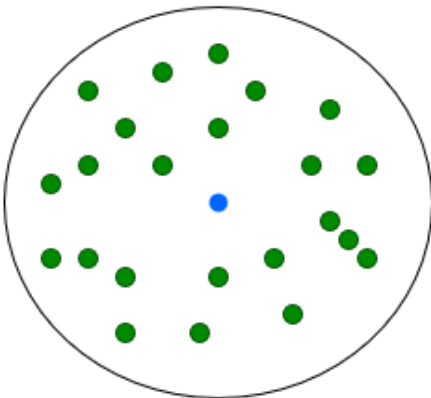
Cluster 1 $\sim \sqrt{N}$ followers



Cluster 2 $\sim \sqrt{N}$ followers



Cluster 3 $\sim \sqrt{N}$ followers



Το αντεστραμμένο ευρετήριο αποθήκευεται σε δομή δεδομένων dictionary της Python όπου κάθε κλείδι αντιστοιχεί με μια συστάδα και κάθε value τα key values των αντικειμενων του πίνακα S

```

#inverted index creation
def invertedindex(S,train,nofclusters):
    #κανουμε train τον kmeans με τη χρηση του training set που περιεχει το συνολο δεδομενων
    centroids,_=kmeans2(train,nofclusters,init='points')
    #το inverted index ειναι δομης dictionary οπου για καθε κλειδι υπαρχει μια λιστα που περιεχει ολα τα
    #κλειδια απο το συνολο S...δηλαδη δεν αποθηκευει τις τιμες αλλα τα indexes του S
    invertedindex={i: [] for i in range(nofclusters)}

    chunks=train.shape[0]

    #χωρισμος των δεδομενων σε chunks για πιο ευκολη επεξεργασια
    for i in range(0,S.shape[0],chunks):
        #τα πρωτα i μετα τα επομενα i + chunks ...
        chunk=S[i :i+chunks]
        #εχοντας αποθηκευσει τα κεντρα βρισκουμε τα κοντινότερα κεντρα για καθε chunks δεδομενων
        distances=cdist(chunk,centroids,metric='euclidean')

        #closest centroids
        nearest=np.argmin(distances,axis=1)
        #μετραμε τα κοντινότερα διανυσματα και τα τοποθετουμε ολα σε ενα cluster για καθε cluster θα εχουμε ~1000 διανυσματα/συσταδα
        for j, centroid_id in enumerate(nearest):
            invertedindex[centroid_id].append(i + j)

    #μετατρεπουμε καθε λιστα σε np array
    for i in range(nofclusters):
        invertedindex[i]=np.array(invertedindex[i],dtype=np.int32)

    return centroids,invertedindex

```

Ενδεικτικό screenshot της μορφής των συσταδων

```

cluster: 997
value: [ 3500  3565  3569 ... 997392 997856 998456]
cluster: 998
value: [ 1052  1065  1794 ... 997932 999468 999888]
cluster: 999
value: [ 1457  1496  1893 ... 998966 999251 999416]

```

Ακριβής Αλγόριθμος

Για την ακριβή προσέγγιση η λύση του προβλήματος είναι πολύ απλή: Ελέγχουμε όλα τα δεδομένα του συνόλου S με κάθε ερώτημα $q \in Q$ και βρίσκουμε τα k κοντινότερα διανύσματα του συνόλου S με τη χρήση της ευκλείδειας απόστασης.

```
def PreciseNearestNeighbors(q,index:dict, centroids:np.ndarray,S:np.ndarray,k):
    results=[]
    basekeys=np.concatenate(list(index.values()))

    computations=len(q)*len(basekeys)

    for i in range(len(q)):
        distances=np.linalg.norm(S[basekeys]-q[i],axis=1)

        vectorindices=np.argsort(distances)[:k]

        results.append(basekeys[vectorindices])

    return results,computations
```

Παίρνουμε τα κλειδιά απ

Ο αλγόριθμος είναι πολύ ακριβής και βγαζει σχεδόν πάντα υψηλό recall score

$$(\text{Euclidean distance} = \text{Norm}(x - y) = \|x-y\| = \sqrt{x-y} = \sqrt{(x_1-y_1)^2 + (x_2-y_2)^2 + \dots + (x_m-y_m)^2})$$

Οπου $x, y \in \mathbb{R}^m$ διανύσματα m διαστάσεων

Πολυπλοκότητα αλγορίθμου

$O(L)$ (for l in range $\text{len}(q)$) * $O(N)$ (distance calculation for each vector of dataset S) = $O(N * L)$

N : Μέγεθος του συνόλου S

L : Μέγεθος του συνόλου Q

Για 1,000,000 Διανύσματα S (N) και 10,000 Διανύσματα $Q(L)$ Ο αλγόριθμος Θα κάνει 10,000,000,000 υπολογισμούς

Προσεγγιστικός αλγόριθμος

Λογική πίσω απο τον προσεγγιστικό αλγόριθμο:

Για καθε $q \in Q$ εξετάζει τις M ($M < \text{sum}(\text{clusters})$) πιο υποσχόμενες ομάδες όπου με μεγαλύτερη πιθανότητα θα περιέχονται οι κοντινότεροι γείτονες του q .

έπειτα υπολογίζουμε την ευκλείδια απόσταση μεταξύ των q και όλων των διανυσμάτων καθε M συστάδας.

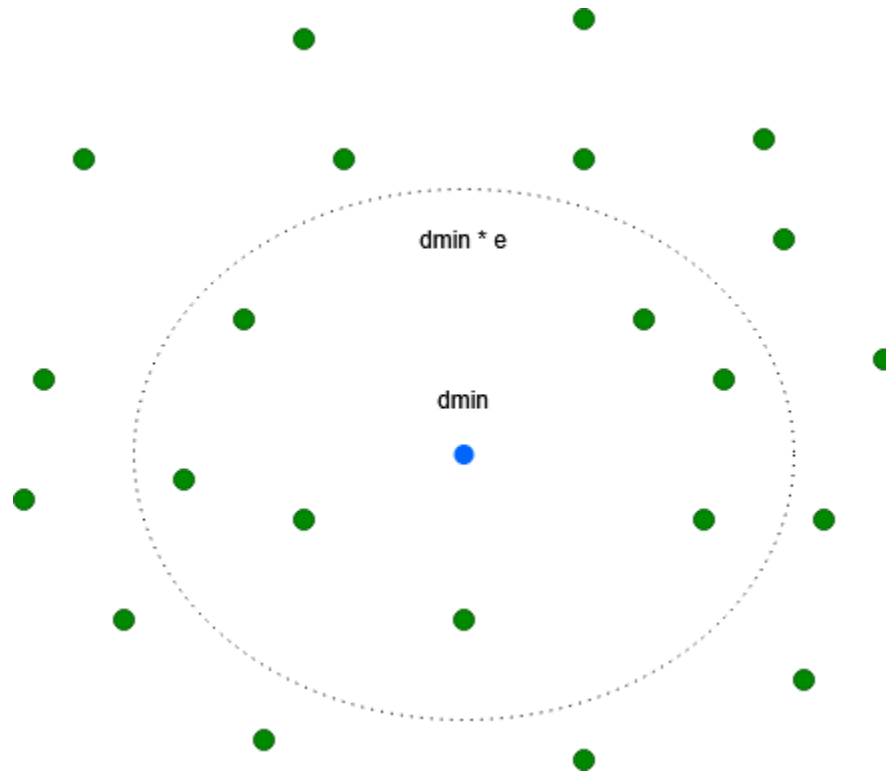
- Τιμή του M : Μεταβλητή τιμή που αλλάζει για κάθε query

1+ ϵ Approximate Nearest Neighbors:

Για κάθε διάνυσμα υπολογίζουμε όλες τις αποστάσεις που έχουν με τα κέντρα των συστάδων και βρίσκουμε τη κοντινότερη συστάδα. Ορίζουμε μία σταθερά $\epsilon > 1$

η οποία καθορίζει το περιθώριο ανοχής κατά την επιλογή συστάδων. Το μέγιστο επιτρεπτό όριο απόστασης ορίζεται ως $d_{\min} * \epsilon$

Αν για καθε συστάδα ισχύει ότι $d(\text{centroid}, q) \leq (d_{\min} * \epsilon)$ τότε η συστάδα αυτή θεωρείται αρκετά κοντινή και επιλέγεται. Τέλος, τα διανύσματα όλων των επιλεγμένων συστάδων συνενώνονται για να σχηματιστεί το τελικό σύνολο ελέγχου. Στον κώδικα το ϵ εχει οριστεί ως 1.5, δηλαδή θα επιλεχθούν οι συστάδες που έχουν μεχρι και 1.5 φορές μεγαλύτερη του $d_{\min}$ απόσταση απο το ερώτημα χρήστη



```
def ApproximateNearestNeighbors(q:np.ndarray,index:dict,centroids:np.ndarray,S:np.ndarray,k,e=1.5):
    approxresults=[]
    computations=0
    for i in range(len(q)):

        clusterdistances=np.linalg.norm(centroids-q[i],axis=1)

        sortedclusters=np.argsort(clusterdistances)

        dmin=clusterdistances[sortedclusters[0]]

        #λογική πίσω από την αυτοματοποίηση:
        #1+ε Approximate nearest neighbor search για τα κοντινότερα clusters
        #ορίζουμε dmin το κοντινότερο διάνυσμα στο query μας
        #ορίζουμε ένα tolerance ratio=1.5,αυτό σημαίνει
        #για κάθε cluster ελέγχουμε αν ισχύει η ανισότητα d(q,c(m)) <= e * dmin
        #αν ισχύει τότε εισαγούμε το cluster στην αναζήτηση, δηλαδή M = M+1

        clusters=[]
        for clustid in sortedclusters:
            dist=clusterdistances[clustid]
            if dist > (dmin * e): break

            clusters.append(clustid)
```

Τέλος, κάνουμε K Nearest Neighbors στα διανύσματα των επιλεγμένων συστάδων

```
#cluster index
basekeys=np.concatenate([index[c] for c in clusters])

computations+=len(basekeys)

vectorsforcheck=S[basekeys]

distances=np.linalg.norm(vectorsforcheck-q[i],axis=1)

vectorindices=np.argsort(distances)[:k]
approxresults.append(basekeys[vectorindices])
return approxresults,computations
```

Πολυπλοκότητα αλγόριθμου:

N: Μέγεθος του συνόλου S

L: Μέγεθος του συνόλου Q

c: αριθμός κοντινότερων συστάδων

$O(L) \text{ (for } l \text{ in range len(q))} * O(c) \text{ (Number of clusters to be checked)} =$

$O(L * c)$

Όμως ξέρουμε ότι clusters = 1000, επομένως

$O(L * 1000) = O(L)$

+ $O(L) * O(N)$ (distance calculation for each vector of dataset S) =

$O(L * N)$ Στη χειρότερη περίπτωση θα γίνει έλεγχος σε όλες τις συστάδες για κάθε ερώτημα

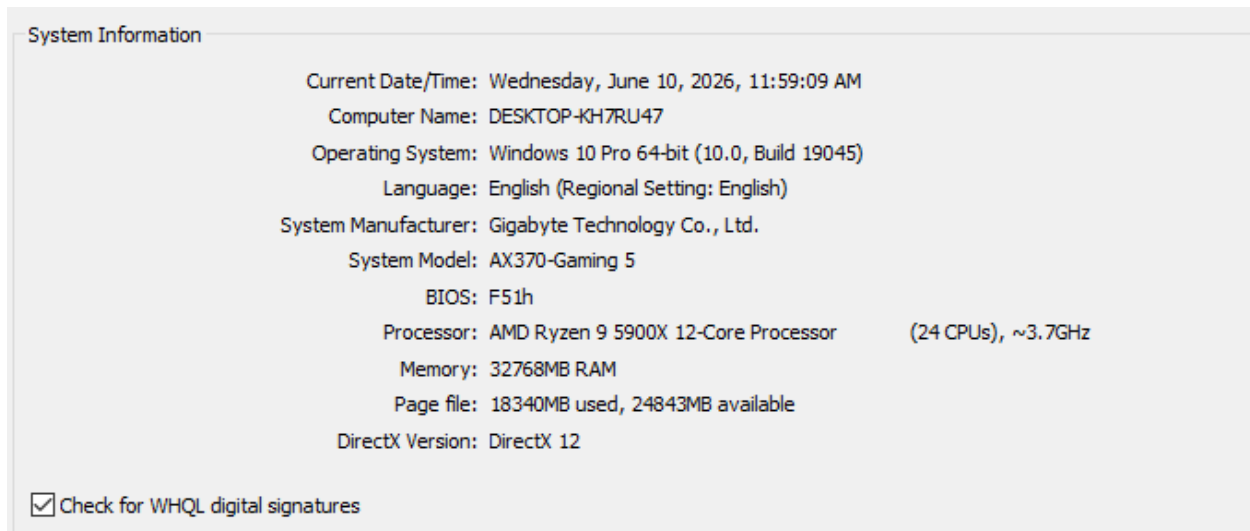
Αρα $O(L * N) + O(L) = O(L * N)$.

(Γενικά για κάθε query ελεγχονται ~500 συστάδες, όπου κάθε μία περιέχει $\sqrt{N}$ διανύσματα, άρα στη μέση περίπτωση ο αλγόριθμος θα κάνει

$$O(L * 500 * \sqrt{N}) = O(L * \sqrt{N}) \text{ επαναλήψεις })$$

Πειραματική Μελέτη Αλγόριθμων

Για την υλοποίηση των πειραμάτων χρησιμοποιήθηκε υπολογιστής με 32gb RAM και επεξεργαστή AMD Ryzen 9 5900X 12 core, μια σημαντική πληροφορία καθώς στο πρόγραμμα γίνονται τρισεκατομμύριες πράξεις ευκλείδειας απόστασης σε διανύσματα 128 διαστάσεων



Χρησιμοποιήθηκαν οι μετρητές:

- Χρόνος εκτέλεσης
- Queries per Second
- Αριθμός πράξεων που πραγματοποιήθηκαν
- Recall

Είναι σημαντικό να σημειωθεί ότι:

- στην μέτρηση χρόνου για τον ANN συμπεριλαμβάνεται και ο χρόνος εύρεσης κοντινότερων συστάδων για κάθε ερώτημα
- Για τον αριθμό πράξεων στον ANN δεν συμπεριλαμβάνεται ο αριθμός πράξεων που έγιναν για την εύρεση κοντινότερων συστάδων, καθώς ο αριθμός αυτός παραμένει σταθερός σε κάθε πράξη και είναι $10,000 * 1000 = 10,000,000$ (queries * clusters). Μπορούμε πολύ εύκολα να τα προσθέσουμε στο τελικό αποτέλεσμα

- Στον κώδικα αποθηκεύονται τα κέντρα και τα αντεστραμμένα ευρετήρια σε 2 εξωτερικά Pickle files, συνεπώς τα αποτελέσματα θα είναι ίδια και για τους 2 αλγόριθμους, εκτός αν ξανατρέξουμε το script Που δημιουργεί τα αρχεία

Για όλα τα ερωτήματα του Q και k=100 οι αλγόριθμοι έβγαλαν τα εξής αποτελέσματα:

```
=====
For approximate nearest neighbors:
Running Time      1184.832 Seconds
Speed             8.4 queries/sec
N of calculations 1,276,418,115
Recall            99.51%
=====
=====
for Precise Nearest Neighbors
Running Time      8967.353 Seconds
Speed             1.1 queries/sec
N of calculations 10,000,000,000
Recall            99.99%
=====
C:\Users\Phaedon\Desktop\information-retrieval-main>
```

Παρατηρήσεις:

- Χρόνος εκτέλεσης: Ο προσεγγιστικός αλγόριθμος, μαζί με την εύρεση κοντινότερων συστάδων ολοκλήρωσε την αναζήτηση μέσα σε 1184 δευτερόλεπτα, δηλαδή ~20 λεπτά, ενώ ο Ακριβής αλγόριθμος ολοκλήρωσε την αναζήτηση σε 8967 δευτερόλεπτα, δηλαδή ~ 2,5 με 3 ώρες. Αυτό μεταφράζεται και στη ταχύτητα των αλγόριθμων όπου ο προσεγγιστικός έλεγχε 8.4 ερωτήματα

το δευτερόλεπτο, σε αντίθεση με τον ακριβή αλγόριθμο που έλεγχε 1.1 ερωτήματα το δευτερόλεπτο

- Μείωση υπολογιστικού φόρτου: Ο προσεγγιστικός αλγόριθμος έκανε για τα ερωτήματα 1,276,418,115 πράξεις, μαζί με των έλεγχο των συστάδων = 1,286,418,115 πράξεις, περίπου το $\frac{1}{10}$ των πράξεων που έκανε ο ακριβής αλγόριθμος
- Recall: εφόσον ο ακριβής έκανε 10,000,000,000 πράξεις μπορούμε να πούμε ότι ο προσεγγιστικός αλγόριθμος έλεγξε περίπου το 12% του συνόλου δεδομένων. Άκομα και με τόσο μικρό ποσοστό ελέγχου δεδομένων ο αλγόριθμος εβγάλε ανάκληση 99.51% σε αντίθεση με τον ακριβή που έβγαλε 99.99%, μια πρακτικά αμνητέα διαφορά, ειδικά αν λάβουμε υπόψη τον χρόνο εκτέλεσης και την ταχύτητα του κάθε αλγόριθμου

Συμπέρασμα

Όπως επιβεβαιώνεται από τα πειραματικά αποτελέσματα, η στρατηγική αναζήτησης σε περιορισμένο σύνολο συστάδων προσφέρει τον ιδανικό υπολογιστικό συμβιβασμό. Επιτρέποντας στον αλγόριθμο να αποκλείσει το 88% των περιπτώσεων δεδομένων, επιτυγχάνεται η δραστική επιτάχυνση αναζήτησης με την ταχύτητα να εκτοξεύεται από 1,1 σε 8,4 ερωτήματα ανά δευτερόλεπτο (QPS). Παράλληλα η διαφορά στο ποσοστό ανάκλησης είναι τόσο μικρή που μπορεί να θεωρηθεί και αμνητέα, με αποτέλεσμα η προσεγγιστική μέθοδος να επιλέγεται ως η κατάλληλη λύση του προβλήματος.

Πηγές αναφοράς

- Google Gemini (Για την ανάκτηση δεδομένων από fvec & ivec αρχεία)
- Numpy Documentation
- Διαφάνειες μαθήματος